

Report: Custom DataLoader Design for CIFAR-10

1. Introduction

In deep learning workflows, the data loading pipeline plays a crucial role in determining training efficiency, model generalization, and experiment flexibility. While PyTorch provides standard datasets and loaders, designing a **custom DataLoader** allows better control over data preprocessing, augmentation, debugging, and experimentation.

This report describes the design and implementation of a **custom CIFAR-10 DataLoader** using PyTorch, highlighting its structure, features, and advantages over a basic loader.

2. Dataset Description

The CIFAR-10 dataset consists of:

- **60,000 RGB images** of size `32 × 32`
- **10 classes** (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
- Split into:
 - **50,000 training images**
 - **10,000 test images**

3. Custom Dataset Design

A custom dataset class `CIFAR10Custom` was implemented by inheriting from `torch.utils.data.Dataset`. This allows full control over how samples are loaded, transformed, and returned.

3.1 Key Design Choices

- **Train/Test Awareness:**
The dataset behaves differently depending on whether it is used for training or testing.
- **Data Augmentation:**
Augmentations are applied only during training to improve generalization.
- **Subset Sampling:**
A configurable fraction of the dataset can be used for faster debugging or controlled

experiments.

- **Optional One-Hot Encoding:**
Labels can be returned either as class indices or one-hot vectors.
- **Debug Visibility:**
Occasional debug prints help verify correct data loading during runtime.

4. Data Preprocessing and Augmentation

4.1 Training Transformations

For the training dataset, the following transformations are applied:

- Random cropping with padding (to increase spatial robustness)
- Random horizontal flipping (to improve invariance)
- Conversion to tensor
- Normalization using CIFAR-10 mean and standard deviation

These augmentations help reduce overfitting and improve model generalization.

4.2 Testing Transformations

For the test dataset, only deterministic preprocessing is applied:

- Conversion to tensor
- Normalization using the same dataset statistics

No augmentation is used during testing to ensure consistent evaluation.

5. Subset Selection

The dataset supports a `subset_fraction` parameter that allows training on a smaller portion of the dataset. This feature is particularly useful for:

- Rapid prototyping
- Debugging model architectures

- Ablation studies

Random sampling ensures the subset remains representative of the full dataset.

6. Label Handling

The custom dataset optionally converts class labels into **one-hot encoded vectors**, which can be useful for:

- Custom loss functions
- Visualization tasks
- Educational and experimental purposes

This flexibility allows the same DataLoader to be reused across multiple experimental setups.

7. DataLoader Configuration

The dataset is wrapped using PyTorch's **DataLoader** with the following settings:

- Mini-batch processing
- Shuffling enabled for training
- Multi-process data loading using worker threads

This ensures efficient GPU utilization and stable training behavior.

8. Advantages of the Custom DataLoader

Compared to a standard CIFAR-10 loader, the custom implementation provides:

1. Greater flexibility in preprocessing and augmentation
2. Easy control over dataset size
3. Built-in debugging visibility
4. Support for multiple label formats

5. Cleaner separation between training and evaluation pipelines

9. Conclusion

The implemented custom DataLoader demonstrates how PyTorch's Dataset and DataLoader abstractions can be extended to build a **modular, flexible, and experiment-friendly data pipeline**. Such customization is essential for real-world deep learning workflows, especially when experimenting with architectures, optimization strategies, and visualization techniques.

This custom DataLoader forms a strong foundation for further tasks such as model training, FLOPs analysis, gradient flow visualization, and experiment tracking using tools like Weights & Biases.

Github link - <https://github.com/JPranav21/-MLOps-JP-M25CSA021.git>

Wandblink-

<https://wandb.ai/pranav-j1414-prom-iit-rajasthan/cifar10-cnn-gradient-flow?nw=nwuserpranavj1414>